

Model Evaluation

Every model has something in mind when they got started. Later they competed with others and found their strengths.

Artificial Analysis gives the benchmark:

<https://artificialanalysis.ai/models>

<https://livebench.ai/#/?highunseenbias=true>

We are going to look into model evaluation in two ways: 1. benchmark provided by this website 2. Code

We have a Jupyter notebook code to evaluate models

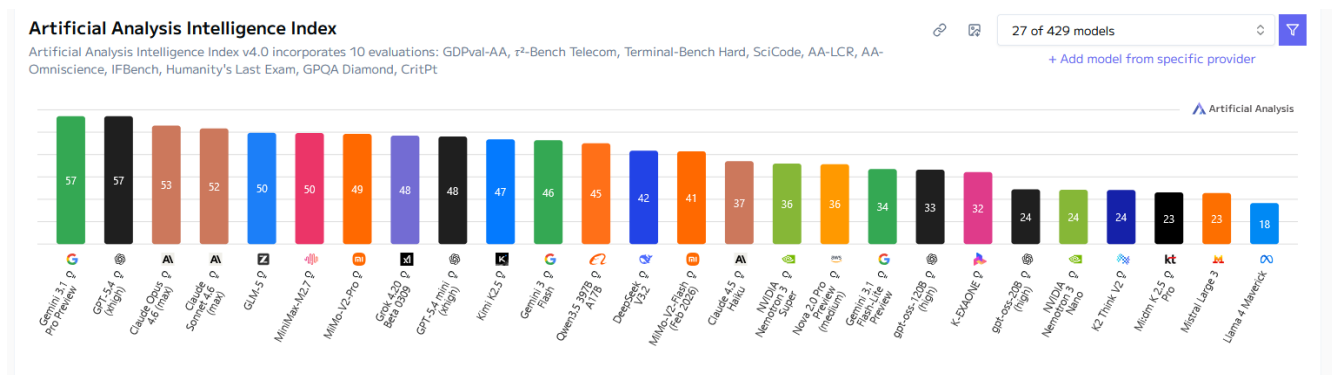
Different models were built for different purposes and we want to evaluate them based on our use-case

Say we want to use the models for solving mathematical problems, thats one use-case

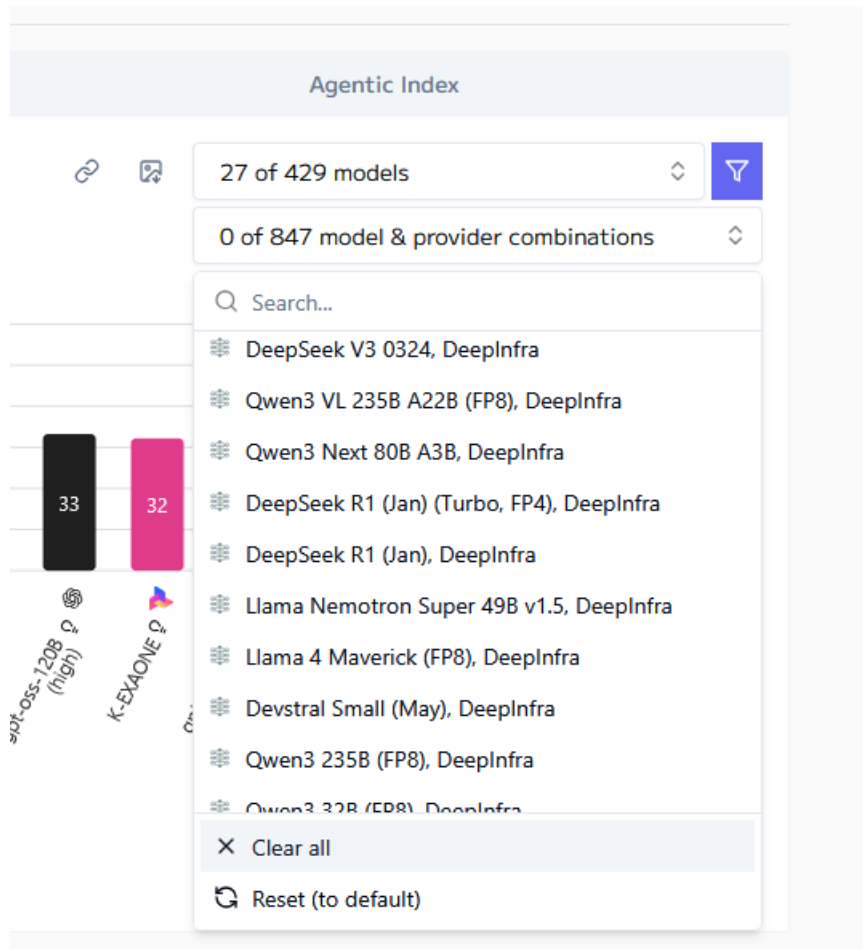
We want a model to be very creative

We want a model to generate code - another use-case

You want to test a model whether it is smarter than humans, for this we give competitive exams to AI

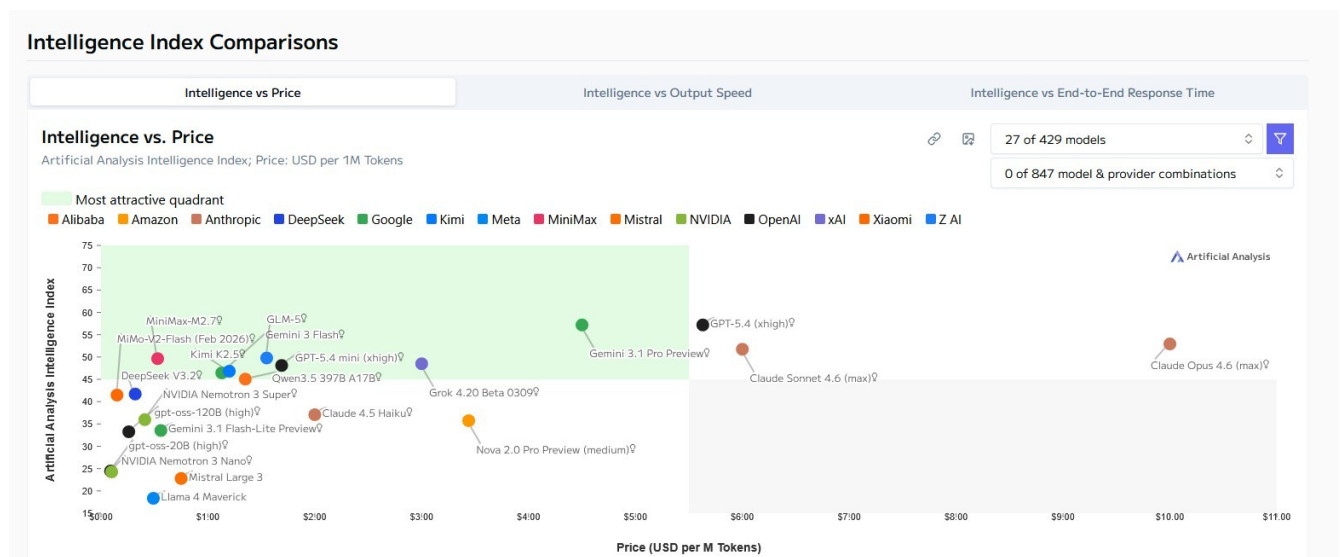


if I want to onboard a new model for comparison, I can do as well



Intelligence vs Price

We need to pick highest intelligence with our available budget



Gemini 3.1 Pro preview might be good

<https://livebench.ai/#/?sort=Coding+Average&highunseenbias=true>

Search...									
Filter by organization...									
Model	Organization	Global Average	Reasoning Average	Coding Average	Agentic Coding Average	Mathematics Average	Data Analysis Average	Language Average	IF Average
GPT-5.2 Codex	OpenAI	74.30	77.71	83.62	51.67	88.77	78.20	73.68	66.45
GPT-5.1 Codex Max	OpenAI	71.95	84.57	81.38	56.67	83.66	54.89	75.39	67.13
Claude 4 Sonnet	Anthropic	50.98	39.67	80.74	38.33	60.36	44.07	71.01	22.68
Claude Sonnet 4.5 Thinking	Anthropic	68.19	77.59	80.36	53.33	79.31	56.97	76.45	53.35
GPT-5.1 Low	OpenAI	59.95	59.64	80.30	40.00	68.38	45.87	77.85	47.62

For coding, best model is GPT-5.2 Codex

This is for Benchmarking

How do we evaluate by our code?

We can write code in Python, which is very easy but Python code is very slow. But AI will help you convert into Rust code, which is very fast. We do entire development in Python, everytime we do that, we convert into Rust. We need to select the best model for this use-case. We have 4 models:

HuggingFace, OpenAI, Anthropic, Google. Now we want to evaluate.

What should be the evaluation steps?

1. Write Python code
2. Run this code
3. Example: count prime numbers up to 1 million. It takes a lot of time in Python say 4.5 seconds
4. Convert Python code into Rust.
5. I will ask each of the 4 models to convert Python code into Rust
6. 4 different models will give us 4 different Rust codes.
7. Run Rust code from each model and calculate time maybe 0.5 seconds
8. How fast Rust code is running is one metric.
9. Second metric is, how correct the output code is?
10. How much time it takes to generate the Rust code?
11. What if we can ask some other LLM to be a judge?
12. Pass the code output(s) into some other LLM to evaluate the model
13. To run Rust code, install Rust compiler as well
14. Will check if the CUDA is running
15. We need to pass in HuggingFace, OpenAI API key, Gemini and Anthropic keys (Anthropic is the judge). All other models are MaaS as opposed to HF, which will use our Colab resources
16. Find benchmark for Python (time it takes to run the code) then for Rust code

```
=====
Gemini 2.5 Flash META-EVALUATION RESULT
=====

Winner Model: Hugging Face - Qwen Coder 1.5B

Model Ranking:
-----
Rank      : 1
Model     : Hugging Face - Qwen Coder 1.5B
Reason    : This model delivered correct, compiling, and runnable Rust code that achieved the fastest execution time (0.271s)
-----
Rank      : 2
Model     : OpenAI - GPT
Reason    : The model produced correct, compiling, and runnable Rust code. It achieved very good performance, with a Rust execution time of 0.274s.
-----
Rank      : 3
Model     : Google - Gemini 2.5 Flash
Reason    : This model generated correct, compiling, and runnable Rust code. While functional, its Rust execution time (0.931s) was significantly slower than the top models.
-----
Rank      : 4
Model     : Anthropic - Claude Sonnet 4.6
Reason    : This model failed to generate any code due to an API error ('Your credit balance is too low'). Consequently, it was ranked last.
-----

Overall Reasoning:
The models were ranked primarily on correctness, then compile/run success, Rust runtime, speedup, and finally code quality/complexity.
=====
```

	model_label	task	python_result	rust_result	python_execution_time_seconds	rust_execution_time_seconds	speedup_vs_python	task_id
0	Hugging Face - Qwen Coder 1.5B	count_primes	78498	78498	8.8909	0.271177	32.7863	tf
1	Google - Gemini 2.5 Flash	count_primes	78498	78498	5.7328	0.931014	6.1576	tf
2	Anthropic - Claude Sonnet 4.6	count_primes	None	None	NaN	NaN	NaN	tf
3	OpenAI - GPT	count_primes	78498	78498	4.4185	0.274618	16.0896	tf